

Reviewer Pack — Minimal Rank of Tropicalized Quiver Representations vs Monot...

Entry f6311e88496f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 07:03:50 UTC

Minimal Rank of Tropicalized Quiver Representations vs Monotone Circuit Depth for k-CLIQUE

Entry ID: f6311e88496f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 07:03:50 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Tropicalization) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For every quiver representation over a field with finite characteristic, the minimal rank of its tropicalized version is at least $\Omega(n^{\{k/4\}})$, where n is the number of vertices and k is the size of the clique.’, ‘This lower bound holds even if we allow the use of NOT gates at the inputs only.’, ‘For the k-CLIQUE indicator, this minimal rank is exactly $\Theta(n^{\{k/4\}})$.’]

Rationale (proposer’s reasoning):

[‘Tropicalization has been successfully used to encode properties of graphs and circuits. Quiver representations are a generalization of graphs and circuits, making them a promising candidate for exploring lower bounds.’, ‘The quiver representation theory provides a framework to study the structure of graphs and circuits in a tropical setting, which might expose new structures that are not visible in classical settings.’, ‘Previous work has shown that tropicalized objects can be used to derive lower bounds on circuit complexity, suggesting that quiver representations could also be useful in this context.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 205aca0c088cae14

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a given quiver representation, the minimal rank of its tropicalized version is at least $\Omega(n^{(k/4)})$ and the corresponding monotone circuit depth is within $\Theta(n^{(k/4)})$, where n is the number of vertices, k is the size of the clique, and the difference between the measured depth and the theoretical bound is ≤ 2 for all seeds. The conjecture is falsified if any seed produces a minimal rank below $\Omega(n^{(k/4)})$ or a monotone circuit depth outside of $\Theta(n^{(k/4)})$ by more than t

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropicalization AND representation theory AND monotone circuit complexity AND k-CLIQUE - minimal rank AND quiver representations AND tropicalized AND $\Omega(n^{(k/4)})$ AND k-CLIQUE - $\Theta(n^{(k/4)})$ AND NOT gates AND minimal rank AND quiver representations AND monotone circuits

Top relevant hits considered: - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2411.19889v1] Tropical representations and valuated matroids - [http://arxiv.org/abs/2410.08349v1] Tropical subrepresentations of the boolean regular representation in low dimension - [http://arxiv.org/abs/1908.03035v2] Twisted Indices of more 3d Quivers - [http://arxiv.org/abs/0711.1135v3] The rank of a quiver representation - [http://arxiv.org/abs/1610.08333v1] Properties of minimal mutation-infinite quivers - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if i != j:
```

```

        factor = A[j][i]
        for k in range(n):
            A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def rank(A):
    A_tilde = gaussian_elimination(A)
    r = sum(1 for row in A_tilde if any(row))
    return r

def monotone_circuit_depth(n, k):
    # Placeholder function to simulate the construction of a monotone circuit
    # This is a dummy implementation and should be replaced with actual logic
    return n + k # Example: depth = n + k

for _ in range(30): # Ensure at least 30 instances per seed
    n, k = random.randint(5, 40), random.randint(2, min(n-1, 5)) # Ensure k < n
    A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    rank_tropicalized = rank(A)
    depth_circuit = monotone_circuit_depth(n, k)

    if rank_tropicalized < n**(k/4) or abs(depth_circuit - n**(k/4)) > 2:
        return {
            "metric_name": "Rank and Depth",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, k={k}, rank_tropicalized={rank_tropicalized}, depth_circuit={depth_circuit}"
        }

    return {
        "metric_name": "Rank and Depth",
        "metric_value": None,
        "instances_tested": 30,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 89)) # Default to first 30 primes if
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_rank = sum(r["metric_value"] for r in results) / len(results)
std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd5aa302.py", line 86, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd5aa302.py", line 59, in run_trial
    n, k = random.randint(5, 40), random.randint(2, min(n-1, 5)) # Ensure k < n
                                         ^
UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16335
2	propose	ollama_remote	glm4:latest	0	0	13396
3	propose	ollama_remote	glm4:latest	0	0	14428
4	propose	ollama_remote	glm4:latest	0	0	13651
5	preregistration	ollama_remote	glm4:latest	0	0	9934

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	novelty	ollama_remote	glm4:latest	0	0	8493
7	novelty	ollama_remote	glm4:latest	0	0	11220
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43100
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9133
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15788
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10535
12	judge	ollama_remote	glm4:latest	0	0	27581

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193595 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in *research/audit/f6311e88496f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f6311e88496f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f6311e88496f.tar.gz* (if generated)